PMPH (2026)
Home Assignment 1

Mathias Nygaard Larsen, xbm145

September 9, 2026

Contents

1 Task 1: Prove stuff

1.1 Prove that a list-homomorphism induces a monoid structure

In order to prove that $(\text{Img}(h), \circ)$ is a monoid, we want to prove that $\circ$ is associative and e is the neutral element.
First let's prove associativity. I follow the hint, so if we consider that h is of the form $h : A \to B$, then $\text{Img}(h) = \{h(a) \mid a \in A\}$. This means that for any three $x, y, z \in \text{Img}(h)$, there also exists $a, b, c \in A$ such that $h(a) = x, h(b) = y$ and $h(c) = z$.
To prove associativity, we want to prove that for any three x, y, z it holds that $(x \circ y) \circ z = x \circ (y \circ z)$.
Starting from x, y, z we can apply the hint and get:

$$\begin{aligned}(x \circ y) \circ z &= (h(a) \circ h(b)) \circ h(c) \\ &= h(a + +b) \circ h(c) \\ &= h((a + +b) + +c) \\ &= h(a + +(b + +c)) \\ &= h(a) \circ h(b + +c) \\ &= h(a) \circ (h(b) \circ h(c)) \\ &= x \circ (y \circ z)\end{aligned} \tag{1}$$

where we apply $(x = h(a)), (y = h(b))$, and $(z = h(c))$ for the first and last equalities. The second, third, fifth, and sixth equalities follow from the homomorphism property $h(u + +v) = h(u) \circ h(v)$ used in either direction. And finally we use the associativity of list concatenation $(a + +b) + +c = a + +(b + +c)$. This means that $\circ$ is associative on $\text{Img}(h)$.
Now let's prove that for all $x \in \text{Img}(h)$ we have that $x \circ e = e \circ x = x$
Consider any $x \in \text{Img}(h)$, then we have that there exists an a such that $h(a) = x$. we also know from the first rule of LH that $e = h([])$. Starting with $x \circ e$, we have:

$$\begin{aligned}x \circ e &= h(a) \circ h([]) \\ &= h(a + +[]) \\ &= h(a) \\ &= x\end{aligned} \tag{2}$$

where we insert the definition of x and e, use the third rule to obtain $h(a + +[])$ and use that concatenation with the empty list is neutral, and convert back to $h(a) = x$. Now this only shows that $x \circ e = x$, so we also need to show $e \circ x = x$. This argument is symmetrical.
Again taking any $x \in \text{Img}(h)$, we have that there exists an a such that $h(a) = x$.
Starting from $e \circ x$:

$$\begin{aligned} e \circ x &= h([]) \circ h(a) \\ &= h([] + +a) \\ &= h(a) \\ &= x \end{aligned} \tag{3}$$

Now we have shown both directions, and hence we have proved that $x \circ e = e \circ x = x$. Based on these two things, we've proved that $(\text{Img}(h), \circ)$ is a monoid.

1.2 Prove the Optimized Map-Reduce Lemma

We follow the hint from the assignment description, and apply the rule $(\text{reduce}(++)[]) \circ (\text{distr}_p\ x) == x$ for any list x. Since this is just the identity, we can insert it:

$$\begin{aligned} (\text{reduce}(+)0) \circ (\text{map}f) &= (\text{reduce}(+)0) \circ (\text{map}f) \circ (\text{reduce}(++)[]) \circ (\text{distr}_p) \\ &= (\text{reduce}(+)0) \circ (\text{reduce}(++)[]) \circ (\text{map}(\text{map}f)) \circ (\text{distr}_p) \\ &= (\text{reduce}(+)0) \circ (\text{map}(\text{reduce}(+)0)) \circ (\text{map}(\text{map}f)) \circ (\text{distr}_p) \\ &= (\text{reduce}(+)0) \circ (\text{map}((\text{reduce}(+)0) \circ (\text{map}f))) \circ (\text{distr}_p) \end{aligned} \tag{4}$$

Here we use Theorem 3 in section 2.4.2 of the course notes [**?**]
In the first equality, we insert the identity $(\text{reduce}(++)[]) \circ (\text{distr}_p)$. In the second equality, we use lemma 2 to move $(\text{map}f)$ across $(\text{reduce}(++)[])$. In the third equality, we use lemma 3 with $\odot = +$ and $e * \odot = 0$. Finally, we use lemma 1 to combine the two consecutive map operations into $(\text{map}((\text{reduce}(+)0) \circ (\text{map}f)))$. This is what we wanted to show.

2 Task 2: Longest Satisfying Segment

2.1 Implementation

The parallel LSS implementation represents each segment by the tuple$(lss, lis, lcs, len, first, last)$. To combine two segments x and y, we first determine whether the concluding satisfying segment of x can be connected to the initial satisfying segment of y. The empty segment must also be handled since it is used as the neutral element of the reduction. We implemented the missing values as:

```
let segments_connect = x_len == 0 || y_len == 0 || pred2 x_last y_first

let new_lss = max (max x_lss y_lss) (if segments_connect then x_lcs + y_lis else 0)
let new_lis = if x_lis == x_len && segments_connect then x_lis + y_lis else x_lis
let new_lcs = if y_lcs == y_len && segments_connect then x_lcs + y_lcs else y_lcs
let new_len = x_len + y_len
```

The longest satisfying segment after combining two segments is therefore either the longest segment in x, the longest segment in y, or a segment crossing the boundary between x and y.
I tested this implementation for the three provided test files `same`, `zeros`, and `sorted`. The tests passed for both the C and CUDA backends.

2.2 Validation tests

To ensure that LSSP works as intended, I've added a few validation tests to the existing files.
For **lssp_same.fut**, the following validation tests are added:

```
-- input { [1i32, 1, 1, 2, 2, 3] }
-- output { 3 }
--
-- input { [4i32, 4, 4, 4, 4] }
-- output { 5 }
```

For **lssp_sorted.fut**, the following validation tests are added:

```
-- input { [3i32, 1, 2, 3, 0] }
-- output { 3 }
--
-- input { [1i32, 2, 3, 4, 5] }
-- output { 5 }
```

And finally, for **lssp_zeros.fut**, the following tests were added:

```
-- input { [1i32, 0, 0, 0, 2, 0] }
-- output { 3 }
--
-- input { [0i32, 0, 0, 0, 0] }
-- output { 5 }
--
```

All tests pass and return the expected output, so I've concluded my implementation is correct.

2.3 Performance

For the performance evaluation, I use random arrays containing 10^6, 10^7, and 10^8 `i32` elements. I then compared the provided sequential implementation with the parallel implementation. Speedup is calculated as T_{seq}/T_{CUDA}, and the performance can be seen on Table 1

Predicate	Elements	Sequential (μs)	CUDA (μs)	Speedup
same	10^6	2,017	73	27.6×
same	10^7	20,165	210	96.0×
same	10^8	115,654	1,035	111.7×
zeros	10^6	2,790	77	36.2×
zeros	10^7	24,137	179	134.8×
zeros	10^8	160,516	1,147	139.9×
sorted	10^6	7,466	75	99.5×
sorted	10^7	74,243	174	426.7×
sorted	10^8	511,992	1,036	494.2×

Table 1: Runtime and speedup of the parallel CUDA implementation compared with the sequential C implementation.

The CUDA implementation is way faster than the sequential implementation for all predicates on the large datasets. The speedup also generally increases with the input size. For example, for `sorted`, the speedup increases from approximately $\approx 100\times$ for 10^6 elements to $\approx 500\times$ for 10^8 elements. This is consistent with larger inputs providing enough parallel work to better utilize the GPU, while the overhead associated with parallel execution is less significant for larger inputs
The exact speedups differ between the predicates because the amount of work performed by the sequential implementation is different. In particular, `sorted` has a substantially slower sequential implementation than `same`, while their CUDA runtimes are similar. The larger speedup for `sorted` should not be interpreted as the GPU itself being approximately five times faster for `sorted` than for `same`, much of the difference comes from the different sequential baseline.

3 Task 3: CUDA Vector Addition

For this task I implement vector addition $C[i] = A[i] + B[i]$ using both a CPU implementation and a parallel CUDA implementation. The CPU implementation iterates sequentially over all elements, and the CUDA implementation assigns one vector element to each GPU thread.

3.1 CUDA implementation

The CUDA kernel is shown below:

```
__global__ void vecAddGPU(float* A, float* B, float *C, unsigned int N) {

    const unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

Each thread computes its global index using its block index, block size and thread index. The condition `i < N` ensures that threads in the final block which is only partially filled do not access elements outside the vectors.
I decided to use 256 threads per block. The number of blocks is computed by rounding the required number of threads up to the nearest complete block:

```
unsigned int block_size = 256;
unsigned int grid_size = (N + block_size - 1) / block_size;

vecAddGPU<<<grid_size, block_size>>>(d_A, d_B, d_C, N);
```

This allows the implementation to handle any vector sizes. The previous implementation from Lab-1 was restricted to a maximum length 1024, which corresponds to the maximum number of threads in a single CUDA block.

3.2 Validation

I follow the details in the assignment description, and compare the GPU results element-wise with the sequential CPU result. I used $\epsilon = 10^{-6}$, and the implementation is considered valid if $|C_{\text{CPU}}[i] - C_{\text{GPU}}[i]| < 10^{-6}$
for every element. All three tested datasets successfully passed validation.

3.3 Performance measurements

I measured CPU time using a single execution of the sequential implementation, and GPU time is measured by executing the CUDA kernel 300 times. The timing excludes GPU memory allocation and memory transfers. The reported CUDA runtime is the average runtime per kernel execution.
The memory throughput is computed from the total amount of memory accessed during the vector addition. Each element requires two reads and one write, so we have $3 \cdot 4 = 12$ bytes per element.
So the memory throughput is computed as

$$\text{Throughput} = \frac{12N}{T_{\text{GPU}}}, \tag{5}$$

where T_{GPU} is the CUDA execution time. The measured results are shown in Table 2.
The medium dataset achieves the largest speedup, almost 200× while the large dataset achieves approximately 100×. This does not mean that the GPU performs worse for the

Dataset	N	CPU (μs)	CUDA (μs)	Speedup	GB/s
Small	20,057	58	4.94	11.75×	48.75
Medium	1,000,031	2014	10.24	196.74×	1172.29
Large	100,000,009	81423	888.86	91.60×	1350.04

Table 2: Runtime, speedup and memory throughput for the sequential CPU and CUDA vector addition implementations.

large dataset. We can see that the measured GPU memory throughput increases from 1172 GB/s for the medium dataset to 1350 GB/s for the large dataset.
The difference is because speedup is defined as $S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$
and therefore depends on how the CPU and GPU execution times scale relative to each other. For the medium dataset, the CUDA execution time is small relative to the sequential CPU time, producing a large speedup. For the large dataset, both execution times increase, but their relative scaling results in a smaller speedup despite the higher GPU memory throughput.

4 Task 4: Flat Sparse Matrix-Vector Multiplication

The sparse matrix is represented by a flat array `mat_val`, containing pairs of column indices and non-zero values, and the array `mat_shp`, which contains the number of non-zero elements in each row.
The flat-parallel implementation is:

```
let spMatVctMult [num_elms][vct_len][num_rows]
                 (mat_val: [num_elms](i64, f32))
                 (mat_shp: [num_rows]i64)
                 (vct: [vct_len]f32)
                   : [num_rows]f32 =

  let shp_sc = scan (+) 0 mat_shp
  let flags = mkFlagArray mat_shp false (replicate num_rows true) :> [num_elms]bool
  let prods = map (\(i,x) -> x * vct[i]) mat_val
  let sums = sgmSumF32 flags prods
  let res = map (\i -> sums[i-1]) shp_sc
  in res
```

The first line, `let shp_sc = scan (+) 0 mat_shp`, computes the cumulative row sizes. For example, if `mat_shp = [2,3,3,2,1]`, then `shp_sc = [2,5,8,10,11]`. These values indicate where each row ends in the flattened representation.
The flag array is constructed using `mkFlagArray`. It marks the beginning of each row in the flattened matrix. For the example above, the flags are `[T,F,T,F,F,T,F,F,T,F,T]`.

Next, all matrix-vector products are computed independently using the flat `map`. Each pair (i, x) in `mat_val` contains the column index i and the non-zero matrix value x. The matrix value is multiplied by the corresponding value `vct[i]` in the dense vector.
The products are then accumulated within each row using the segmented scan `let sums = sgmSumF32 flags prods`. The last value in each segment is the dot product for that row.
Finally, these values are extracted using `let res = map (\i -> sums[i-1]) shp_sc`. Since `shp_sc` contains the positions immediately after the end of each row, the final value of each segment is found at index `i-1`.

4.1 Correctness

I added an extra test with irregular row sizes. The sparse matrix i added corresponds to

$$A = \begin{bmatrix} 0 & 0 & 7 & 0 \\ 2 & -1 & 3 & 4 \\ 0 & 5 & 0 & 0 \end{bmatrix}, \qquad x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}.$$

The row sizes are `mat_shp = [1,4,1]`, so the test checks that rows with different numbers of non-zero elements are handled correctly.
The expected result is

$$Ax = \begin{bmatrix} 7 \cdot 3 \\ 2 \cdot 1 - 1 \cdot 2 + 3 \cdot 3 + 4 \cdot 4 \\ 5 \cdot 2 \end{bmatrix} = \begin{bmatrix} 21 \\ 25 \\ 10 \end{bmatrix}.$$

The reference test used in the code is:

```
-- ==
-- compiled input {
--   [2i64, 0i64, 1i64, 2i64, 3i64, 1i64]
--   [7.0f32, 2.0f32, -1.0f32, 3.0f32, 4.0f32, 5.0f32]
--   [1i64, 4i64, 1i64]
--   [1.0f32, 2.0f32, 3.0f32, 4.0f32]
-- }
-- output { [21.0f32, 25.0f32, 10.0f32] }
```

The reference test passed using the CUDA backend.

4.2 Performance

I measured performance on a sparse matrix with 10,000 rows and 1,000,000 non-zero elements. Each row contains 100 non-zero elements. The same input was used for the sequential and flat-parallel implementations, and both implementations were run 10 times.

Implementation	Runtime (μs)	Speedup
Sequential Futhark	1894.2	1.00×
Flat-parallel CUDA	207.2	9.14×

Table 3: Runtime and speedup for sparse matrix-vector multiplication with 1,000,000 non-zero elements and 10,000 rows.

The speedup is calculated as $S = T_{\text{seq}}/T_{\text{CUDA}}$.
The flat-parallel implementation is about 9.1× faster than the sequential implementation for this dataset. The products for the non-zero matrix elements can be computed in parallel, while the segmented scan combines the products belonging to each row.

References